

Section - A

Ans 1: Abstraction is the process of hiding complex implementation details and showing only essential features of a system.

we use it because:

- Reduce system complexity
- Makes design and development easier
- Enable modularity.

Ans 2: Levels of Abstraction:

1. Hardware Level - Physical component (CPU, memory)
2. Machine level - Binary instructions (0s and 1s)
3. Assembly Level - Symbolic, instruction (Add, Mov)
4. High-Level language - C, Java, Python
5. Application Level - End user programs.

Example:

Printing "Hello" → Python code → compiled to assembly → converted to machine code → executed by hardware.

Ans 3: Functional units:

- Input unit: Takes input from user
- Memory unit: stores data and instructions.
- Arithmetic Logic unit (ALU): Performs calculations.
- Control unit (CU): controls execution of instructions
- Output unit: Display result

ALU vs CU:

- ALU - performs arithmetic and logical operations
- CU - controls and coordinates all operations.

Register vs cache:

- Registers - smallest, fastest storage inside CPU.
- cache - slightly larger, stores frequently used data.

Ans 5: Von - Neumann Model:

- A compute architecture where CPU, memory, and I/O are connected and instructions, data share the same memory.

stored-program concept:

- Instructions are stored in memory like data and executed sequentially.

Ans 6: Limitation caused: due to a single path (bus) between CPU and memory.

Key points:

- Data and instructions share same bus
- causes delay in execution
- limits system performance.

Ans 7: Number of transistors on a chip doubles approximately every (8-24 months)

Impact:

- Increased Performance
- Reduced cost
- smaller and more powerful devices.

- Q8: IPC (Instruction per cycle)
- IPC is the number of instructions a CPU executes in one cycle.
- Higher IPC $\rightarrow$ Better CPU Performance.

Ans 9: Execution time: Total time to run a program.

Clock Rate: Number of cycles per second

CPI (cycle per instruction): Avg cycle per instruction.

IPC: Instruction per cycle

Relation: $\text{Execution time} = \frac{\text{Instructions} \times \text{CPI}}{\text{Clock Rate}}$

$$\text{IPC} = 1/\text{CPI}$$

Ans 10: Definitions:

Bit: Smallest unit (0 or 1)

Byte: 8 bits

Signed: Represents positive and negative

Unsigned: Only positive values

Example:

8-bit unsigned: 0 to 255

8-bit signed: -128 to +127

Ans 11: Basic operations are:

- Arithmetic (addition, subtraction, multiplication)
- Logical (AND, OR, NOT)
- Comparison ($>$, $<$, $=$)
- Data movement (Load/store)

Ans 12: Character Representation:

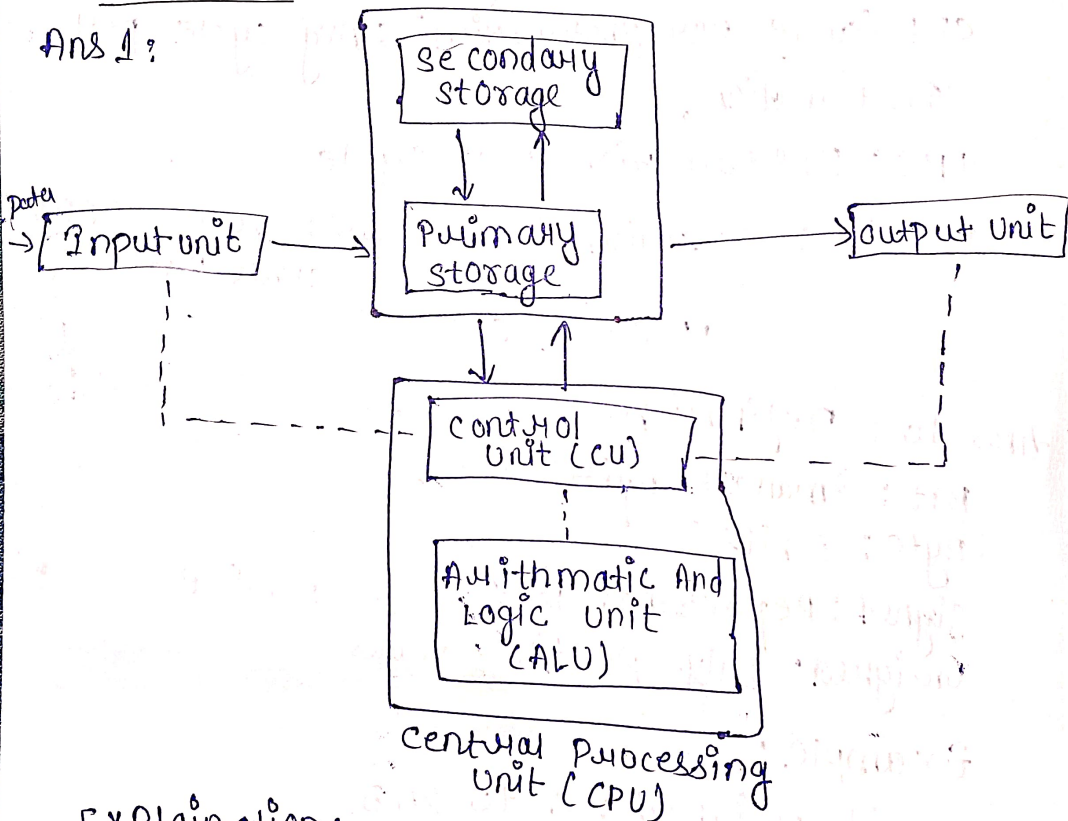
- ASCII (7/8-bit)
- Unicode (supports multiple languages)

Decimal Representation:

- Fixed-Point (simple representation)
- Floating-Point (scientific, e.g. IEEE 754)

Section-B

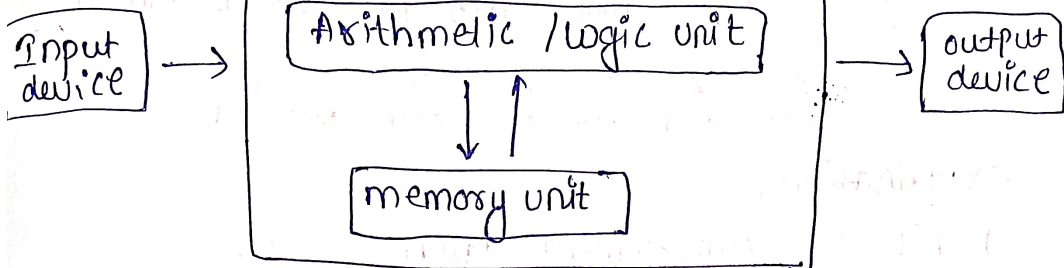
Ans 1:



Explanation:

- Input unit takes data, CPU processes it using ALU and CU.
- Memory storage data and instructions
- Output unit display results; buses connect all unit.

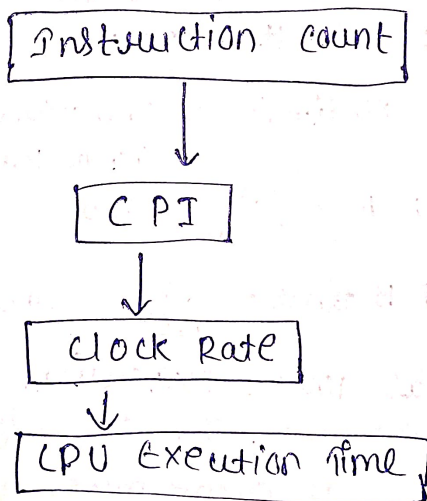
Q2:



Explanation

- Single memory stores both data and instructions.
- CPU communicates with memory and I/O via a common bus.
- Basis of most modern computers.

Ans 3:



$$IPC = 1 / CPI$$

Explanation:

- CPU time depends on instruction count, CPI and clock rate.
- Higher clock rate $\downarrow$ time, higher CPI $\uparrow$ time.
- IPC is inverse of CPI (higher IPC = better performance)

Ans 4: Diagram (Range)

Decimal :	-8	-7	-6	-5	-4	-3	-2	-1	0
Binary :	1000	1001	1010	1011	1100	1101	1110	1111	0000
	1	2	3	4	5	6	7		
	0000	0001	0010	0010	0011	0100	0101		

Explanation:

- 4-bit 2's complement range -8 to +7
- MSB (leftmost bit) indicates sign
- used to represent negative numbers efficiently

Section - C

Ans 1: Given

CPU A : 3.4 ITZ, CPI 1.5

CPU B : 2.29 ITZ, CPI 1.0

$$\text{Formula: CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

$$\text{Now, CPU A} = \frac{1.5}{3.0} = 0.5$$

$$\text{CPU B time} = \frac{1.0}{2.2} = 0.45$$

∴ CPU B has lower execution time, hence faster
↳ Reason: lower CPI outweighs lower clock rate

Ans 2: Effect of cache miss:

- cache miss → CPU waits (stall cycles increases)
- IPC decreases (fewer instructions per cycles)
- execution time increase

fix for it (any one):

- increase cache size
- improved cache locality
- use multi-level cache

Given,

Range : 200 Values

Precision: 0.1 $\rightarrow$ need decimal accuracy

Step 1: Choice

- use fixed Point Representation

Step 2: Justification

- fixed - Point is sufficient for limited range.
- More efficient and faster than floating - Point.
- Less memory usage and simple hardware.

Ans 4: Critique of given statement are mentioned below:

$\rightarrow$ Moore's law refers to transistor count, not direct performance.

$\rightarrow$ performance gains now limited due to:

- $\rightarrow$ power and heat constraints
- $\rightarrow$ slower clock speed scaling

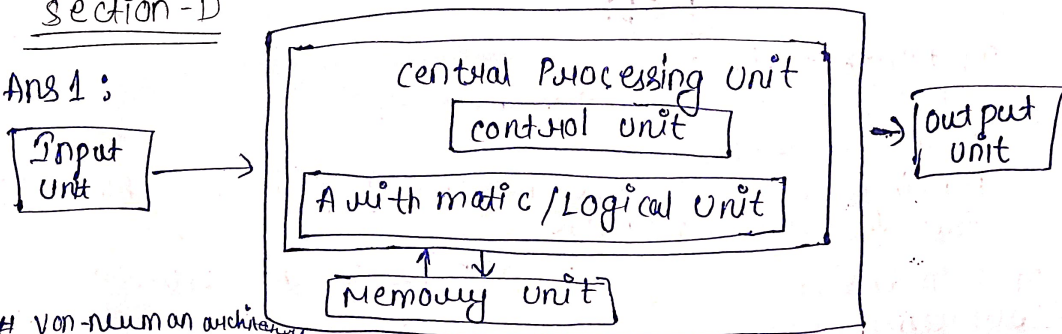
$\rightarrow$ Modern improvement depends on:

- $\rightarrow$ parallelism (multi-core CPUs)
- $\rightarrow$ Better architecture, not

Conclusion Moore's Law alone does not guarantee automatic performance doubling today.

Section - D

Ans 1:



Von-Neuman architecture

Design Decision:

- single memory for data and instructions.

Explanation:

- Simplify system design and reduces hardware cost.
- But leads to von-neumann bottleneck: due to shared bus.

Ans 2: Performance Relation:

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock Rate}}$$

Explanation:

- Performance improves when CPU time decreases.
- Depends on IC, CPI and clock rate

One practical improvement:

- Reduce CPI using pipeline or better cache
- Reduce IC using optimized algorithm
- Increase clock rate using better hardware.

Ans 3: Represent Number:

Let's take +5 and +4 (4-bit system)

→ +5 → 0101

→ +4 → 0100

$$\begin{array}{r} \text{Addition: } 0101 \\ + 0100 \\ \hline 1001 \end{array}$$

Interpretation:

1001 in 2's complement = -7

overflow check

- Adding two positive numbers → result should be positive.
- But result is negative → overflow occurs.

In conclusion, overflow happens when result exceeds representable range.